

Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет прикладної математики

Кафедра системного програмування і спеціалізованих комп'ютерних систем

Лабораторна робота № 1

з дисципліни

«Моделювання та оптимізація комп'ютерних систем»

Тема: «Ознайомлення з пакетом MATLAB. Базові конструкції мови програмування»

Виконала студентка групи:

КВ-11 Михайліченко Софія

Перевірив: Малезик П.М.

Оцінка:

Київ – 2024

Мета роботи:

Ознайомлення з операційним середовищем системи **MATLAB**.

Командне вікно системи **MATLAB**.

Редактор **M**-файлів.

Робоча область.

Основні команди.

Програмування в системі **MATLAB**.

Завдання 1

1. Нехай задано 2 вектори.

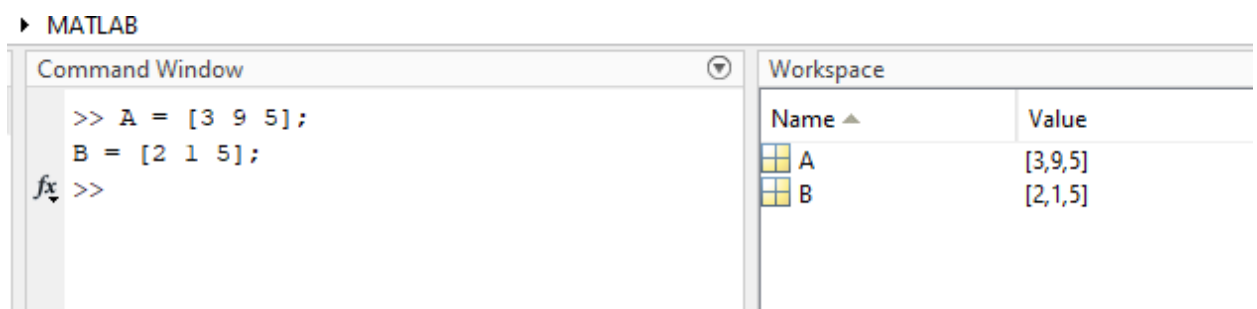
$A = [3 \ 9 \ 5];$

$B = [2 \ 1 \ 5];$

Виконання завдання 1

>> A = [3 9 5];

>> B = [2 1 5];



Задаємо у MATLAB вектори A і B, використовуючи квадратні дужки для створення масивів. У даному випадку, вектор A та B має три елементи.

Після цього вектори A і B будуть доступні для використання в програмі.

Важливо замітити, якщо оператор закінчується символом ';', результат його дії не відображується в командному вікні, а, якщо запис оператора не закінчується символом ';', результат дії цього оператора одразу ж виводиться в командне вікно.

Command Window	Workspace						
<pre>>> A = [3 9 5]; B = [2 1 5]; >> A = [3 9 5] B = [2 1 5]</pre>	<table> <tr> <th>Name ▲</th><th>Value</th></tr> <tr> <td>A</td><td>[3,9,5]</td></tr> <tr> <td>B</td><td>[2,1,5]</td></tr> </table>	Name ▲	Value	A	[3,9,5]	B	[2,1,5]
Name ▲	Value						
A	[3,9,5]						
B	[2,1,5]						
<pre>A = 3 9 5 B = 2 1 5</pre>							

Завдання 2

2. Обчислити оператори:

$$C = A ./ B.^2$$

$$C = (A ./ B).^2$$

$$B = \text{sqrt}(A(2)) + 2 * B(1)$$

Виконання завдання 2

Спочатку проаналізуємо обчислення, які нам потрібно виконати.

У першому використовується поелементне ділення векторів, воно здійснюється за допомогою сукупності знаків " ./ ". Результат – вектор, кожний елемент якого є часткою від ділення відповідного елемента першого вектора на відповідний елемент другого вектора. Також використовується поелементне піднесення до степеня, здійснюється за допомогою сукупності знаків " .^ ". Результат – вектор, кожний елемент якого є відповідним елементом першого вектора, піднесеним до степеня, розмір якого дорівнює значенню відповідного елемента другого вектора.

У другому обчисленні використовуються ті самі операції, але з іншою послідовністю, спочатку виконується ділення, а потім піднесення до степеня.

У третьому прикладі потрібно взяти корінь з другого елемента вектора A та додати помножений на 2 перший елемент вектору B.

► MATLAB

Command Window		Workspace									
<pre>>> A = [3 9 5]; B = [2 1 5]; >> A = [3 9 5] B = [2 1 5] A = 3 9 5 B = 2 1 5 >> C = A./B.^2 C = 0.7500 9.0000 0.2000 >> C = (A./B).^2 C = 2.2500 81.0000 1.0000 >> B=sqrt (A(2)) + 2*B(1) B = 7</pre>		<table border="1"><thead><tr><th>Name ▲</th><th>Value</th></tr></thead><tbody><tr><td>A</td><td>[3,9,5]</td></tr><tr><td>B</td><td>7</td></tr><tr><td>C</td><td>[2.2500,81,1]</td></tr></tbody></table>		Name ▲	Value	A	[3,9,5]	B	7	C	[2.2500,81,1]
Name ▲	Value										
A	[3,9,5]										
B	7										
C	[2.2500,81,1]										

Завдання 3

3.Порівняти два масиви, використовуючи умову $A < B$

$A = [2 \ 7 \ 6 ; 9 \ 0 \ -1 ; 3 \ 0.5 \ 6] ;$

$B = [8 \ 0.2 \ 0 ; -3 \ 2 \ 5 ; 4 \ -1 \ 7] ;$

Виконання завдання 3

У MATLAB ми можемо порівняти два масиви за допомогою умови $A < B$, щоб отримати логічний масив, де кожен елемент буде true(1), якщо відповідний елемент масиву A менший за відповідний елемент масиву B, і false(0) в іншому випадку. Для цього нам спочатку потрібно визначити

матриці A та B:

Variables - B

3x3 double

	1	2	3	4
1	8	0.2000	0	
2	-3	2	5	
3	4	-1	7	
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				

Workspace

Name	Value
A	[2,7,6;9,0,-1;3,0.5000,6]
B	[8,0.2000,0;-3,2,5;4,-1,...]
C	[2.2500,81,1]

Command Window

2.2500 81.0000 1.0000

>> B=sqrt (A(2)) + 2*B(1)

B =

7

>> A= [2 7 6 ; 9 0 -1 ; 3 0.5 6] ;

B= [8 0.2 0; -3 2 5; 4 -1 7] ;

fx >>

Далі ми застосуємо умову $A < B$:

Variables - result

3x3 logical

	1	2	3	4
1	1	0	0	
2	0	1	1	
3	1	0	1	
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				

Workspace

Name	Value
A	[2,7,6;9,0,-1;3,0.5000,6]
B	[8,0.2000,0;-3,2,5;4,-1,...]
C	[2.2500,81,1]
result	3x3 logical

Command Window

>> result = A < B

result =

3x3 logical array

1 0 0

0 1 1

1 0 1

fx >>

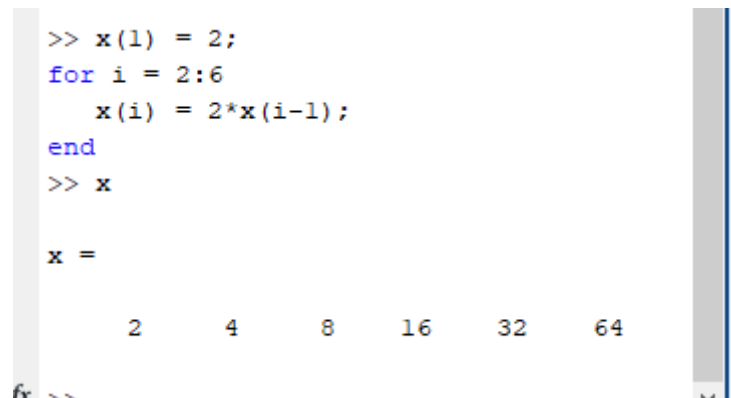
Після виконання цієї операції ми отримаємо логічний масив `result`, де `true` вказує на те, що елемент масиву `A` менший за відповідний елемент масиву `B`, і `false` в іншому випадку. У цьому результаті значення `true` вказують на те, що відповідні елементи масиву `A` менші за відповідні елементи масиву `B`, наприклад, в першому рядку перший елемент масиву `A` (2) менший за перший елемент масиву `B` (8), тому `result(1,1)` дорівнює `true`.

Завдання 4

4. Обчислити підпрограму

```
x(1) = 2;  
for i = 2:6  
    x(i) = 2*x(i-1);  
end
```

Виконання завдання 4



```
>> x(1) = 2;  
for i = 2:6  
    x(i) = 2*x(i-1);  
end  
>> x  
  
x =  
  
     2     4     8    16    32    64
```

Ця підпрограма у MATLAB виконує обчислення значень масиву `x` на основі попереднього значення за таким правилом: кожен наступний елемент масиву `x` дорівнює подвоєному попередньому елементу.

Давайте розглянемо кожен крок виконання цього коду:

1. Початкове значення: $x(1) = 2$; Встановлюється перший елемент масиву `x` як 2.
2. Цикл `for`: `for i = 2:6`. Цей цикл працює з індексами від 2 до 6 (включно).
3. У кожній ітерації циклу виконується наступне: $x(i) = 2 \cdot x(i-1)$; Це означає, що кожен наступний елемент масиву `x` буде дорівнювати подвоєному значенню попереднього елементу.

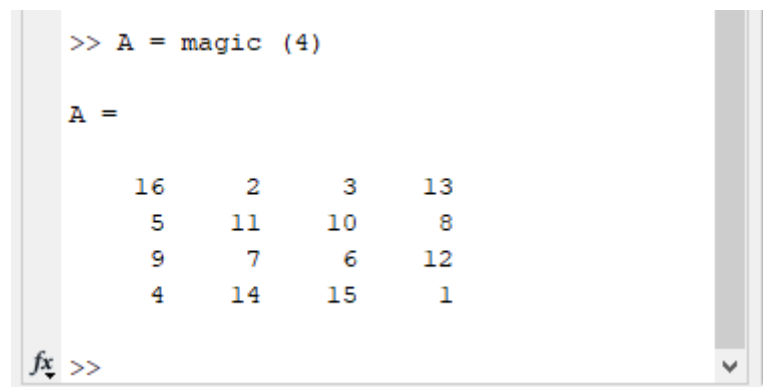
4. Перша ітерація ($i = 2$): $x(2) = 2 \cdot x(1)$; Оскільки $x(1)$ вже встановлено як 2, $x(2)$ буде $2 \cdot 2 = 4$.
5. Друга ітерація ($i = 3$): $x(3) = 2 \cdot x(2)$; Оскільки $x(2)$ було встановлено на попередньому кроці як 4, $x(3)$ буде $2 \cdot 4 = 8$.
6. Такий самий процес повторюється для кожної ітерації циклу для $i = 4$, $i = 5$, $i = 6$.

Завдання 5

5. Обчислити оператор

$A = \text{magic}(4)$

Виконання завдання 5



```
>> A = magic (4)

A =

    16     2     3    13
     5    11    10     8
     9     7     6    12
     4    14    15     1
```

Функція `magic(4)` в MATLAB створює магічний квадрат порядку 4. Магічний квадрат - це квадратна матриця, в якій сума елементів у кожному рядку, кожному стовпці і на кожній з двох головних діагоналей однакова.

Отже, результатом виклику `A = magic(4)` буде створення магічного квадрата розміром 4×4 , який буде збережений у змінній `A`.

Магічний квадрат розміром 4×4 містить числа від 1 до 16 (включно), тому кожен рядок і кожний стовпець має суму 34 ($1+2+\dots+16=136$, і сума кожного рядка або стовпця - $136/4 = 34$).

Завдання 6

6. Створити М – файл, використовуючи текстовий редактор.

```
function C = myfile (a,b)
```

```
C = sqrt ((a.^2) + (b.^2))
```

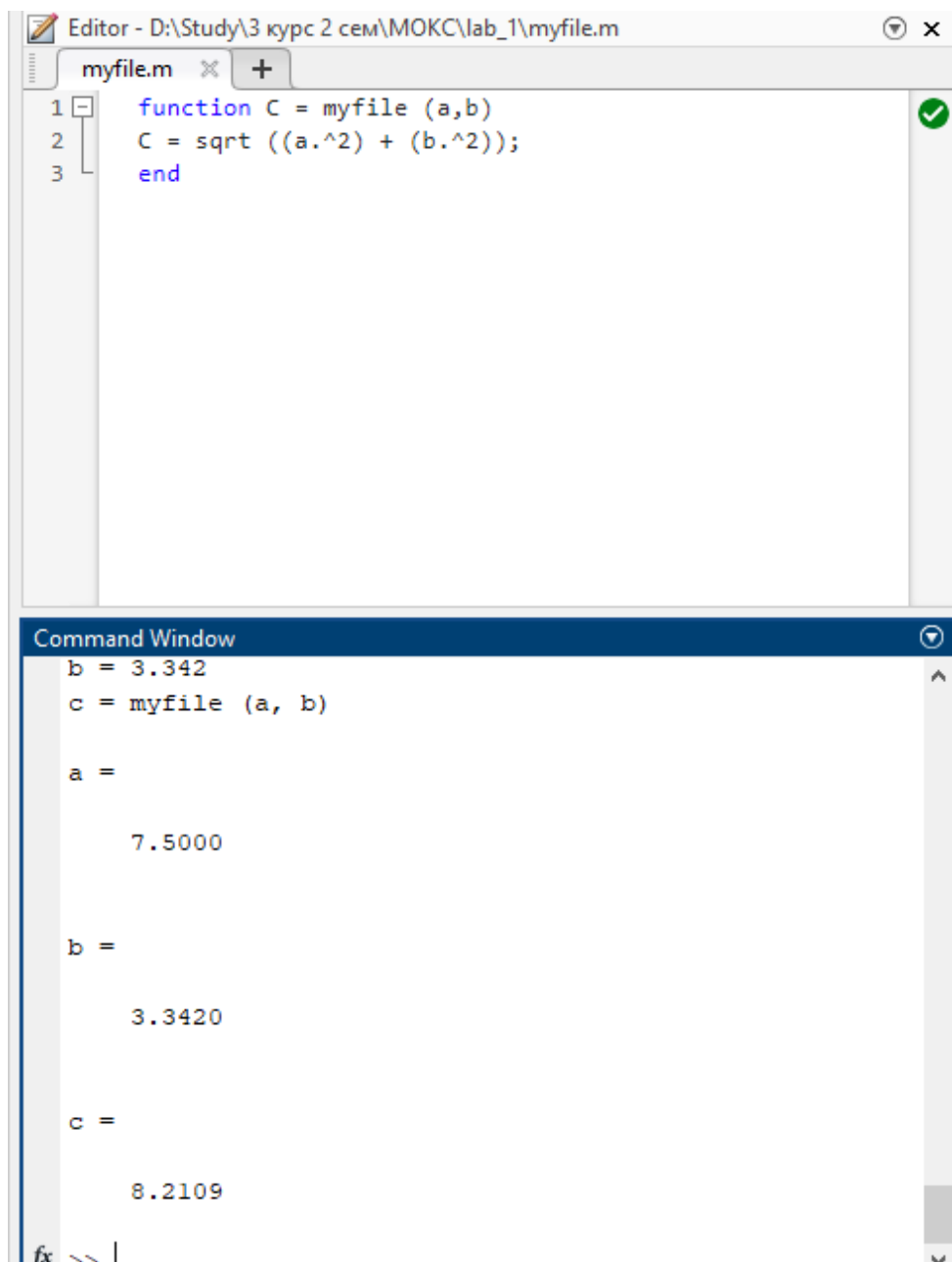
Визвати М – файл із командного рядка або із другого М – файла

```
a = 7.5
```

```
b = 3.342
```

```
c = myfile (a, b)
```

Виконання завдання 6



The screenshot displays the MATLAB environment. The top window is the 'Editor' showing a file named 'myfile.m' with the following code:

```
1 function C = myfile (a,b)
2   C = sqrt ((a.^2) + (b.^2));
3   end
```

The bottom window is the 'Command Window', which shows the execution of the code entered in the Command Window:

```
b = 3.342
c = myfile (a, b)

a =

    7.5000

b =

    3.3420

c =

    8.2109
```

The Command Window prompt is 'fx >> |'.

Для виконання завдання 6 нам потрібно було створити М-файл, використовуючи текстовий редактор:

 myfile.m – Блокнот

Файл Правка Формат Вид Справка

```
function C = myfile (a,b)  
C = sqrt ((a.^2) + (b.^2));  
end
```

Потім ми відкрили створений файл за допомогою MATHLAB та у командному рядку виконали виклик цього файлу, що спричинило виконання написаного коду, а саме піднесення чисел a та b до квадрату, а потім взяття кореня з отриманої суми двох квадратів заданих чисел і вивели результат.

У висновку до цього завдання зазначимо, що М-файли у MATLAB використовуються для збереження скриптів і функцій. Основні причини використання М-файлів замість написання коду безпосередньо в командному рядку:

- 1) **Організація коду:** М-файли дозволяють організувати код у логічні блоки, такі як функції, класи, скрипти та інші фрагменти. Це допомагає підтримувати чистоту коду та зрозумілість програми.
- 2) **Перевикористання коду:** Код можна використовувати багаторазово, викликаючи функції або запускаючи скрипти. Це забезпечує можливість перевикористання коду та вбудовування його у інші програми.
- 3) **Тестування коду:** Код, написаний у М-файлі, може бути легко тестований та налагоджуваний. Можна викликати функції з конкретними аргументами або запускати скрипти, щоб перевірити їхню правильність та ефективність.
- 4) **Документація коду:** У М-файлах можна додавати коментарі та документацію, що полегшує розуміння коду і збільшує його читабельність.
- 5) **Робота з великими проектами:** При розробці великих програм часто використовуються багато М-файлів для розділення коду на логічні модулі та його кращої організації. Це полегшує спільну роботу декількох

розробників над проектом та підтримку програми в подальшому.

Завдання 7

7. Сформувати тривимірний масив нормально розподілених випадкових чисел розміром 4 x 3 x 2.

$B \approx \text{rand}(4, 3, 2)$

Виконання завдання 7

```
>> B = rand (4, 3, 2)

B(:, :, 1) =

    0.8147    0.6324    0.9575
    0.9058    0.0975    0.9649
    0.1270    0.2785    0.1576
    0.9134    0.5469    0.9706

B(:, :, 2) =

    0.9572    0.4218    0.6557
    0.4854    0.9157    0.0357
    0.8003    0.7922    0.8491
    0.1419    0.9595    0.9340

fx >>
```

`rand(4, 3, 2)` - Ця команда викликає функцію `rand()` і передає їй три аргументи. У цьому випадку, перший розмір (4) вказує кількість рядків, другий розмір (3) - кількість стовпців, і третій розмір (2) - кількість глибини (або "висота") тривимірного масиву. Таким чином, отримується тривимірний масив розміром 4 x 3 x 2. Кожен елемент цього масиву буде випадковим числом.

Завдання 8

8. Необхідно задати синусоїду 10 точками і виконати інтерполяцію, використовуючи рідку сітку.

$x = 0:10;$

$y = \sin(x);$

$xi = 0:0.25:10;$

$yi = \text{spline}(x,y,xi);$

$\text{plot}(x,y, 'o', xi, yi, 'b'), \text{grid}$

Виконання завдання 8

```
>> x = 0:10;  
    y = sin(x);  
    xi = 0:0.25:10;  
    yi = spline(x,y,xi);  
    plot (x,y,'o',xi,yi,'b'), grid
```

Ці команди в MATLAB виконують інтерполяцію синусоїди та побудову графіка за допомогою кубічного сплайну на більш рідкій сітці точок.

$x = 0:10;$ - генерує вектор x зі значеннями від 0 до 10 з кроком 1. Цей вектор буде використаний для задання точок по осі x .

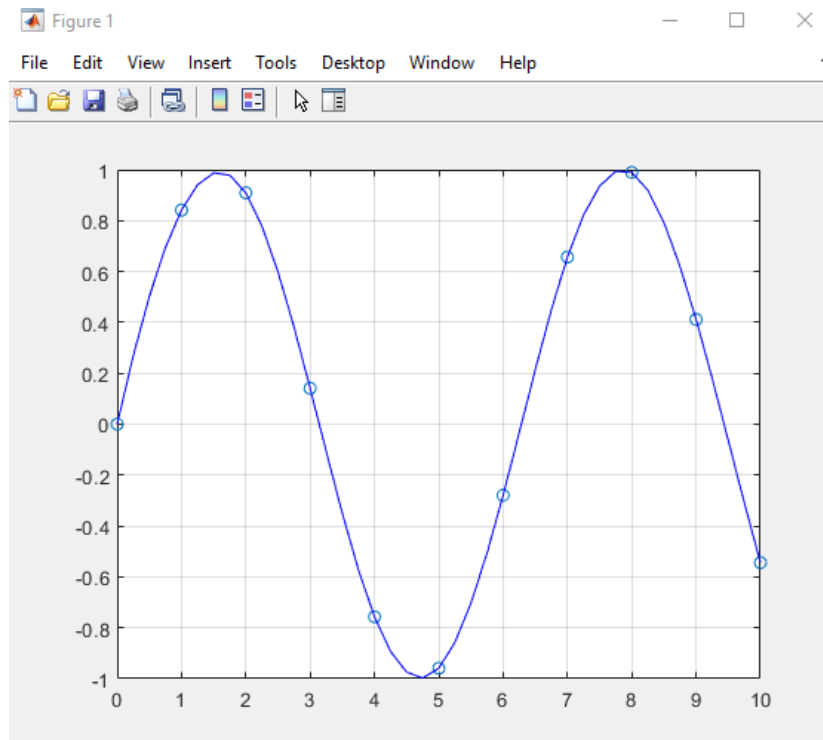
$y = \sin(x);$ - обчислює значення синусоїди для кожного елементу вектора x . Отримані значення зберігаються у векторі y .

$xi = 0:0.25:10;$ - генерує вектор xi зі значеннями від 0 до 10 з кроком 0.25. Цей вектор представляє собою більш рідку сітку точок по осі x , для якої буде виконана інтерполяція.

$yi = \text{spline}(x, y, xi);$ - виконує інтерполяцію кубічним сплайном. Функція `spline()` отримує вектори x і y як вхідні дані (відомі точки) та вектор xi як сітку нових точок, для яких потрібно знайти інтерпольовані значення.

Результат інтерполяції зберігається у векторі yi .

`plot(x, y, 'o', xi, yi, 'b')` - Функція `plot()` використовується для побудови графіка. У цьому випадку вона відображає вихідні точки x та y як округлі маркери `'o'`, а також інтерпольовану криву, що отримана за допомогою інтерполяції кубічним сплайном, за допомогою ліній з кольором `'b'`.
`grid` - Додає сітку до графіка для кращої візуалізації.



Дослідити роботу трьох(підібраних на власний вибір)функцій

1) Дослідимо досить популярну та важливу функцію **`abs(x)`**, де x - вхідне значення, для якого потрібно обчислити абсолютне значення. Якщо x - вектор або матриця, то **`abs()`** обчислює абсолютне значення для кожного елемента вектора або матриці. Якщо x - комплексне число, то **`abs()`** обчислює модуль цього комплексного числа, тобто відстань від початку координат до точки на комплексній площині, яка представляє це комплексне число. Якщо x - дійсне число, то **`abs()`** просто повертає його значення, оскільки абсолютне значення дійсного числа відповідає його модулю.

Наведемо приклади для кожного випадку використання функції:

```
>> abs (4)

ans =

     4

>> abs (-45)

ans =

    45

>> A = [3 9 5];
>> abs (A)

ans =

     3     9     5

>> A = [3 -9 -5];
>> abs (A)

ans =

     3     9     5

>> abs(3 + 4i)

ans =

     5
```

2) Дослідимо функцію **sign(x)** в MATLAB використовується для отримання знаку числа або масиву чисел, де **x** - вхідне значення або масив значень, для якого потрібно визначити знак.

Якщо **x** - додатне число, то **sign(x)** повертає 1.

Якщо **x** - від'ємне число, то **sign(x)** повертає -1.

Якщо **x** - нуль, то **sign(x)** повертає 0.

Наведемо приклади для кожного випадку використання функції:

```

>> sign(-67)

ans =

    -1

>> sign(67)

ans =

     1

>> sign(0)

ans =

     0

>> sign(A)

ans =

     1     -1     -1

>> A = [3 -9 0];
>> sign(A)

ans =

     1     -1     0

```

Дана функція може бути корисною в таких випадках як аналіз даних — якщо потрібно розділити дані на позитивні, від'ємні та нульові значення, функція `sign()` буде корисною для цього. Для нашої сфери ця функція може використовуватись в умовному програмуванні — створення умов, які залежать від знаку числа. Наприклад, можна застосовувати `sign()` для визначення напрямку руху об'єкта, зміни швидкості тощо.

3) Дослідимо функцію **`rem(x, y)`**, де **`x`** та **`y`** - числа, для яких потрібно обчислити залишок, дана функція в MATLAB використовується для обчислення залишку від ділення двох чисел. Ця функція є дуже схожою до операції `%` у багатьох інших мов програмування.

Особливості використання:

- **`x`** та **`y`** можуть бути як дійсними, так і цілими числами.
- залишок від ділення **`x`** на **`y`** обчислюється за допомогою наступних формул:

1. Якщо x і y є цілими числами: залишок є цілим числом, яке задовольняє умову $x = q * y + r$, де q - цілочисельний частка, а r - залишок.
2. Якщо хоча б одне з чисел x або y є дійсним числом: результат обчислення залишку також є дійсним числом.
3. Коли другий аргумент дорівнює нулю, виникає помилка, оскільки ділення на нуль не визначено. MATLAB повідомить про цю помилку і видасть повідомлення про ділення на нуль.

Наведемо приклад, які продемонструють як поведе себе дана функція в різних випадках:

```
>> rem(10,3)

ans =

     1

>> rem(10.5,3.2)

ans =

    0.9000

>> x = [10 15 20];
y = [3 4 5];
>> rem(x,y)

ans =

     1     3     0

>> rem(10,12)

ans =

    10

>> rem(10,0)

ans =

    NaN

>> rem(0,10)

ans =

     0
```